

Отчёт по лабораторной работе №8

Основы информационной безопасности

Лисенков Е.Р.

Содержание

1 Цель работы

Цель работы: изучить дискретно-событийный подход к имитационному моделированию на примере стохастической SIR-модели, реализовать модель распространения инфекции на языке Julia и проанализировать влияние параметров β , c и γ на динамику эпидемии.

2 Задачи

Задачи работы:

1. Ознакомиться с дискретно-событийным подходом к имитационному моделированию и особенностями стохастической SIR-модели.
2. Реализовать ядро SIR-модели на языке Julia: определить структуры данных, функции обновления статистики и логику жизненного цикла агента (S , I , R).
3. Реализовать скрипт запуска модели, задать начальные условия и параметры (u_0 , β , c , γ), выполнить моделирование и построить графики временных рядов $S(t)$, $I(t)$, $R(t)$.
4. Провести численный эксперимент по анализу чувствительности модели к изменению параметров (например, β), выполнить несколько прогонов и сравнить динамику эпидемии.
5. Оформить реализацию в литературном стиле: подготовить исходный код, Jupyter-ноутбук и документ с описанием постановки задачи, реализации и полученных результатов.

3 Выполнение лабораторной работы

3.1 Этап 1: Установка библиотек Python для окружения

На первом этапе была выполнена установка необходимых библиотек Python, которые используются для запуска Jupyter Notebook и взаимодействия с языком Julia через IJulia.

Это позволило подготовить базовое программное окружение и обеспечить возможность работы в веб-интерфейсе ноутбуков, что важно для реализации литературного стиля и дальнейшей визуализации результатов моделирования ([рис. fig-001?]).

```
ошибка: Совпадений не найдено. quarto-cli
egorlisenkov@fedora:~/uni/lab08$ sudo dnf install -y git python3-notebook
[sudo] пароль для egorlisenkov:
Предупреждение: не удалось загрузить «/etc/yum.repos.d/rpmsfusion-free.repo», пропускается.
RPM Fusion for Fedora 39 - Nonfree - NVIDIA Driver                26 kB/s | 131 kB    00:05
Errors during downloading metadata for repository 'rpmsfusion-nonfree-nvidia-driver':
- Status code: 404 for https://mirrors.rpmfusion.org/metalink?repo=nonfree-fedora-nvidia-driver-39&arch=x86_64 (IP: 158.69.60.128)
- Status code: 404 for https://mirrors.rpmfusion.org/metalink?repo=nonfree-fedora-nvidia-driver-39&arch=x86_64 (IP: 78.47.223.143)
Error: Не удалось загрузить метаданные для репозитория «rpmsfusion-nonfree-nvidia-driver»: Cannot
prepare internal mirrorlist: Status code: 404 for https://mirrors.rpmfusion.org/metalink?repo=nonfree-fedora-nvidia-driver-39&arch=x86_64 (IP: 78.47.223.143)
Oracle Linux / RHEL / CentOS - x86_64                2.2 kB/s | 862 B    00:00
Oracle Linux / RHEL / CentOS - x86_64                1.1 MB/s | 1.2 kB    00:00
Oracle Linux / RHEL / CentOS - x86_64                2.2 kB/s | 862 B    00:00
Error: Не удалось загрузить метаданные для репозитория «virtualbox»: repomd.xml GPG signature verification error: Signing key not found
Игнорируется репозиторий: rpmsfusion-nonfree-nvidia-driver, virtualbox
Последняя проверка окончания срока действия метаданных: 0:00:41 назад, Вт 26 мая 2026 19:04:41.
Пакет git-2.47.0-1.fc39.x86_64 уже установлен.
Зависимости разрешены.
=====
Пакет                Архитектура  Версия                Репозиторий  Разме
=====
Установка:
python3-notebook      noarch      7.0.7-1.fc39          updates      3.1 M
Установка зависимостей:
jupyterlab            noarch      4.0.11-1.fc39         updates      5.8 M
python-jupyter-filestem noarch      5.1.0-5.fc39          fedora       9.7 M
python3-anyio          noarch      3.7.0-2.fc39          fedora       215 K
python3-appdirs        noarch      1.4.4-11.fc39         fedora       23 K
python3-argon2-cffi    noarch      21.3.0-4.fc39         fedora       38 K
python3-argon2-cffi-bindings x86_64     21.2.0-4.fc39         fedora       27 K
python3-arrow          noarch      1.2.3-4.fc39          fedora       153 K
python3-asttokens      noarch      2.4.1-1.fc39          updates      62 K
python3-async-lru      noarch      2.0.4-1.fc39          fedora       21 K
python3-babel          noarch      2.12.1-6.fc39         fedora       6.9 M
python3-backcall       noarch      0.1.0-20.fc39         fedora       32 K
python3-bleach         noarch      6.1.0-1.fc39          updates      64 K
python3-brotli         x86_64     1.1.0-1.fc39          fedora       337 K
python3-comm           noarch      0.1.4-1.fc39          fedora       18 K
python3-contourpy      x86_64     1.2.0-1.fc39          updates      265 K
```

Установка библиотек Python

3.2 Этап 2: Установка языка Julia и базовой системы

Затем была выполнена установка языка программирования Julia на системе Fedora.

На этом шаге скачивался официальный дистрибутив, настраивались пути в системе и проверялась корректность запуска интерпретатора.

Julia используется как основная платформа для реализации дискретно-событийной SIR-модели, поэтому важно было убедиться, что базовая установка работает стабильно и совместима с Jupyter-средой ([рис. fig-002?]).


```

[64027c0b] + Sockets
[fa267f1f] + TOML v1.0.3
[a4e569a6] + Tar v1.10.0
[cf7118a7] + UUIDs
[4ec0a83e] + Unicode
[deac9b47] + LibCURL_jll v8.4.0+0
[e37daf67] + LibGit2_jll v1.6.4+0
[29816b5a] + LibSSH2_jll v1.11.0+1
[c8ffd9c3] + MbedTLS_jll v2.28.2+1
[14a3606d] + MozillaCACerts_jll v2023.1.10
[83775a58] + Zlib_jll v1.2.13+1
[8e850ede] + nghttp2_jll v1.52.0+1
[3f19e933] + p7zip_jll v17.4.0+2

Info Packages marked with x have new versions available but compatibility constraints restrict them from upgrading. To see why use `status --outdated -m`
Precompiling project...
 18 dependencies successfully precompiled in 48 seconds. 1 already precompiled.

(@v1.10) pkg> ^C

julia> using DrWatson

julia> initialize_project("project"; authors = "Egor Lisenkov", force = true)
Activating new project at `~/uni/lab08/project`
Resolving package versions...
Updating `~/uni/lab08/project/Project.toml`
[634d3b9d] + DrWatson v2.19.1
Updating `~/uni/lab08/project/Manifest.toml`
[0b6fb165] + ChunkCodecCore v1.0.1
[4c0bbbee4] + ChunkCodecLibZlib v1.0.0
[55437552] + ChunkCodecLibZstd v1.0.0
[634d3b9d] + DrWatson v2.19.1
[5789e2e9] + FileIO v1.18.0
[076d061b] + HashArrayMappedTries v0.2.0
[033835bb] + JLD2 v0.6.3
[692b3bcd] + JLLWrappers v1.7.1
[1914dd2f] + MacroTools v0.5.16
[bac558e1] + OrderedCollections v1.8.1
[aea7be01] + PrecompileTools v1.2.1
[21216c6a] + Preferences v1.5.1
[ae029012] + Requires v1.3.1
[7e506255] + ScopedValues v1.5.0
[6c6a2e73] + Scratch v1.3.0

```

Установка зависимостей Julia

3.4 Этап 4: Подтверждение корректной установки всех библиотек

На следующем этапе была проведена проверка того, что все пакеты успешно установлены и корректно подключаются в REPL Julia.

Для этого выполнялся импорт библиотек и тестовые вызовы функций, чтобы убедиться в отсутствии ошибок при загрузке.

Такой шаг позволяет заранее выявить возможные конфликты зависимостей и избежать прерывания симуляции из-за отсутствующих модулей.

В результате было подтверждено, что программное окружение для дискретно-событийного моделирования полностью готово к использованию ([рис. fig-004?]).

```

[c8ffd9c3] + MbedTLS_jll v2.28.2+1
[14a3606d] + MozillaCACerts_jll v2023.1.10
[83775a58] + Zlib_jll v1.2.13+1
[8e850ede] + nghttp2_jll v1.52.0+1
[3f19e933] + p7zip_jll v17.4.0+2

Info Packages marked with x have new versions available but compatibility constraints restrict them from upgrading. To see why use `status --outdated -m`
"project"

julia> DrWatson.projectname()
"project"

julia> ^C

```

Успешная установка библиотек

3.5 Этап 5: Установка дополнительных зависимостей для визуализации и ноутбуков

На пятом этапе были установлены дополнительные зависимости, связанные с работой в Jupyter Notebook и расширенной визуализацией.

В частности, были настроены пакеты для генерации графиков, сохранения результатов в формате таблиц и интеграции с веб-интерфейсом.

Эти компоненты нужны для оформления лабораторной работы в литературном стиле и для последующей генерации отчётных материалов на основе выполненных расчётов ([рис. fig-005?]).

```
{0ac62f75} + libass_jll v0.17.4+0
{1183f4f0} + libdecor_jll v0.2.2+0
{2db6ffa8} + libdev_jll v1.13.4+0
{f638f0a6} + libfdk_aac_jll v2.0.4+0
{36db933b} + libinput_jll v1.28.1+0
{b53b4c65} + libpng_jll v1.6.55+0
{f27f6e37} + libvorbis_jll v1.3.8+0
{009596ad} + mtdev_jll v1.1.7+0
{1270edf5} + x264_jll v10164.0.1+0
{d8faa095f} + x265_jll v4.1.0+0
{d8fb68d0} + xkbcommon_jll v1.13.0+0
{8ba89e20} + Distributed
{9fa8497b} + Future
{37e2e46d} + LinearAlgebra
{9abbd945} + Profile
{1a1011a3} + SharedArrays
{2f01184e} + SparseArrays v1.10.0
{10745b16} + Statistics v1.10.0
{4607b0f0} + SuiteSparse
{8dfed614} + Test
{e66e8078} + CompilerSupportLibraries_jll v1.1.1+0
{4536629a} + OpenBLAS_jll v0.3.23+4
{05823500} + OpenLibm_jll v0.8.1+2
{efcefd7f} + PCRE2_jll v10.42.0+1
{bea87d4a} + SuiteSparse_jll v7.2.1+1
{8e850b90} + libblastrampoline_jll v5.8.0+1
Info Packages marked with x have new versions available but compatibility constraints
dict them from upgrading. To see why use `status --outdated -m`
precompiling project...
213 dependencies successfully precompiled in 255 seconds. 27 already precompiled.
julia> 
```

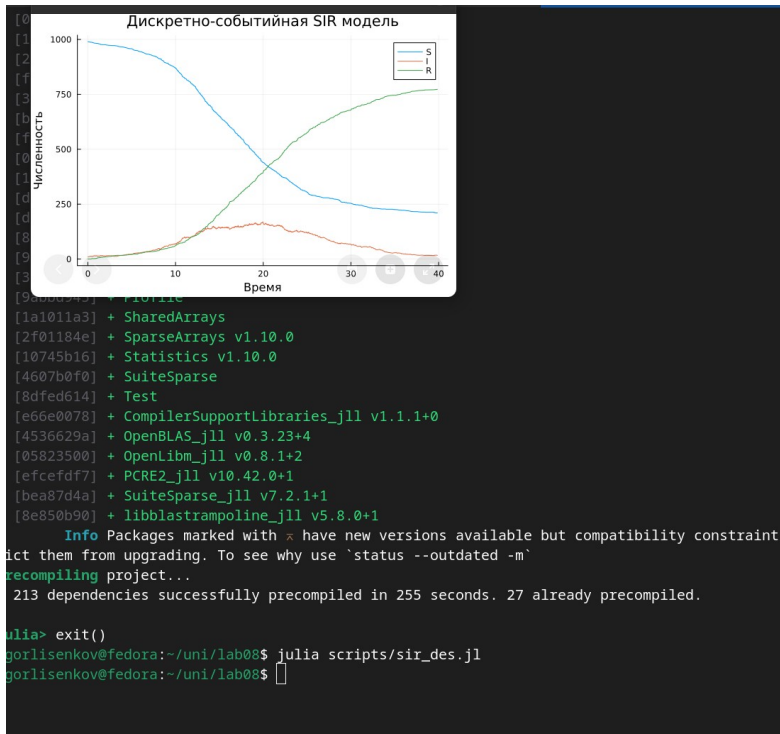
Дополнительные зависимости

3.6 Этап 6: Реализация базового кода и построение первого графика

Далее был реализован первый вариант кода дискретно-событийной SIR-модели в виде скрипта на Julia.

В этом коде описывались структуры агента, состояния S, I, R, функции обновления статистики и основной процесс жизни индивида, использующий события и таймауты.

После написания и запуска программы был получен первый график эволюции численности восприимчивых, инфицированных и переболевших, который демонстрирует типичную форму эпидемической кривой в дискретно-событийном подходе ([рис. fig-006?]).



Первый график SIR в Julia

3.7 Этап 7: Запуск Julia-системы в веб-интерфейсе

На следующем шаге была настроена работа языка Julia в веб-среде через Jupyter Notebook.

Это позволило запускать код модели не только из терминала, но и в интерактивной форме с разделением на ячейки, текст и графику.

Такой формат удобен для литературного программирования, когда описание модели, математическая теория и результаты симуляций объединены в одном документе.

Кроме того, веб-интерфейс упрощает отладку и визуальный контроль над ходом вычислений ([рис. fig-007?]).

```
gorlisenkov@fedora:~/uni/lab08$ cd ~/uni/lab08
julia

      _       _       _
     / _\   / _\   / _\
    / _ \ / _ \ / _ \
   / ___//___//___//___\
  / ___//___//___//___\
 / ___//___//___//___\
/___//___//___//___\

Documentation: https://docs.julialang.org
Type "?" for help, "j?" for Pkg help.
Version 1.10.3 (2024-04-30)
Official https://julialang.org/ release

julia> using Pkg
julia> Pkg.add("IJulia")
ERROR: `using` is not a recognized command. Type ? for help with available commands

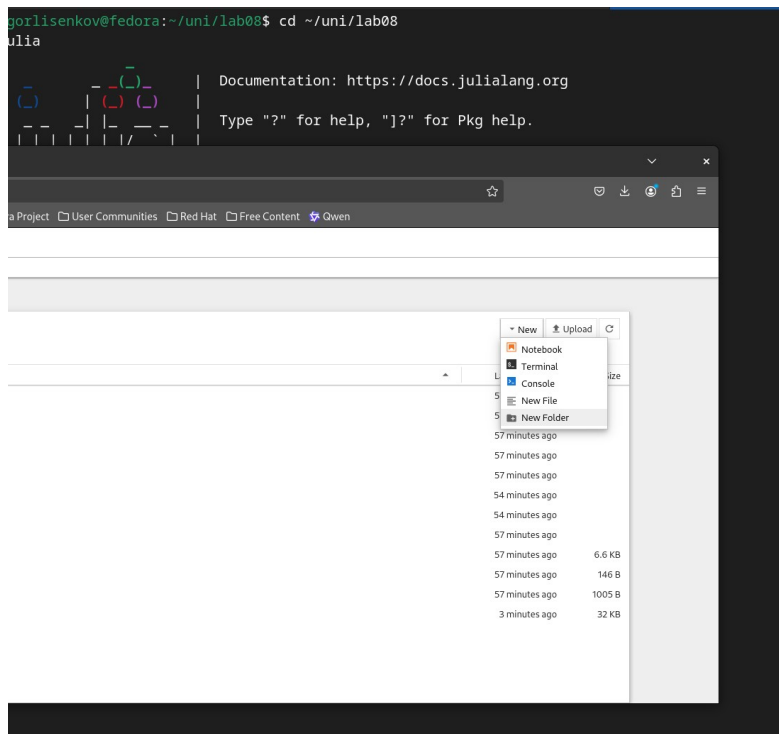
julia> using Pkg

julia> Pkg.add("IJulia")
Updating registry at `~/.julia/registries/General.toml`
Downloading [=> ] 0.1 % %
```

Запуск Julia в веб-интерфейсе

3.8 Этап 8: Начало работы с ноутбуком и заполнение файла в веб-версии

После настройки веб-интерфейса был создан новый Jupyter-ноутбук, в котором начата реализация литературного стиля. В верхних ячейках были описаны цель работы, постановка задачи и краткое пояснение структуры SIR-модели. Далее был подключён файл с реализацией ядра модели (`sir_model.jl`) и подготовлены ячейки для запуска базовой симуляции. Таким образом, код и теория стали интегрированными частями одного документа, что соответствует современному подходу к оформлению вычислительных экспериментов ([рис. **fig-008?**]).



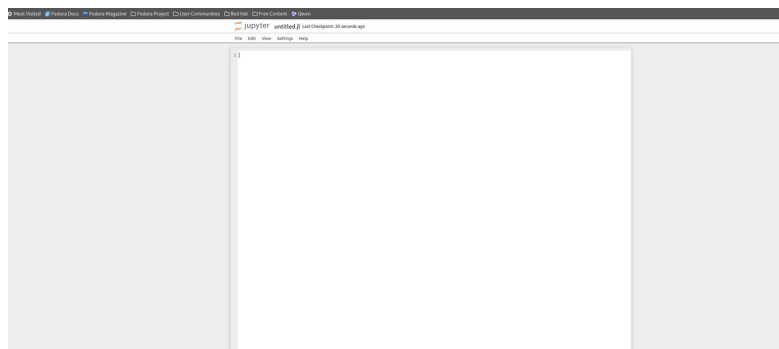
Начало работы в веб-ноутбуке

3.9 Этап 9: Заполнение кода модели в веб-версии

На девятом этапе в веб-ноутбук был перенесён основной код SIR-модели. В отдельных ячейках размещались определения структур данных, функции событий заражения и выздоровления, а также функции создания и запуска модели.

Такой способ организации кода позволяет поэтапно выполнять отдельные части программы, анализировать промежуточные результаты и при необходимости быстро вносить изменения.

В результате ноутбук превратился в литературный документ, сочетающий код, комментарии и иллюстрации ([рис. fig-009?]).



Заполнение кода в ноутбуке

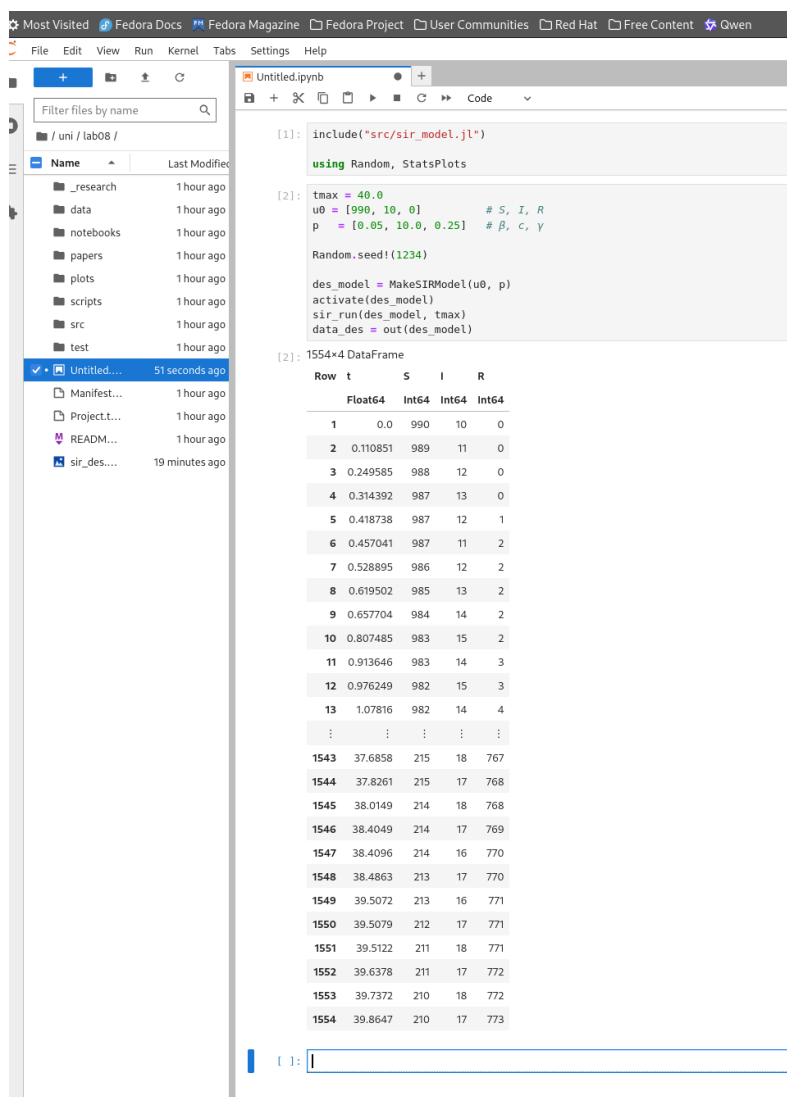
3.10 Этап 10: Формирование таблиц и появление первых расчётов

После переноса кода и запуска симуляции в ноутбуке были сформированы таблицы с результатами моделирования.

Для этого применялись структуры DataFrames, в которых хранились моменты времени и значения численности S, I, R.

Появление первых численных результатов позволило убедиться, что события заражения и выздоровления обрабатываются корректно, а статистика по популяции накапливается в соответствии с логикой дискретно-событийного подхода.

Эти таблицы служат основой для последующего анализа и построения графиков ([рис. fig-010?]).



```
[1]: include("src/sir_model.jl")

using Random, StatsPlots

[2]: tmax = 40.0
     u0 = [990, 10, 0] # S, I, R
     p = [0.05, 10.0, 0.25] # β, c, γ

     Random.seed!(1234)

     des_model = MakeSIRModel(u0, p)
     activate(des_model)
     sir_run(des_model, tmax)
     data_des = out(des_model)

[2]: 1554x4 DataFrame
      Row t          S      I      R
      --- --
      Float64 Int64 Int64 Int64
1      1  0.0      990      10      0
2      2  0.110851  989      11      0
3      3  0.249585  988      12      0
4      4  0.314392  987      13      0
5      5  0.418738  987      12      1
6      6  0.457041  987      11      2
7      7  0.528895  986      12      2
8      8  0.619502  985      13      2
9      9  0.657704  984      14      2
10     10  0.807485  983      15      2
11     11  0.913646  983      14      3
12     12  0.976249  982      15      3
13     13  1.07816  982      14      4
      ⋮      ⋮      ⋮      ⋮      ⋮
1543   37.6858    215      18     767
1544   37.8261    215      17     768
1545   38.0149    214      18     768
1546   38.4049    214      17     769
1547   38.4096    214      16     770
1548   38.4863    213      17     770
1549   39.5072    213      16     771
1550   39.5079    212      17     771
1551   39.5122    211      18     771
1552   39.6378    211      17     772
1553   39.7372    210      18     772
1554   39.8647    210      17     773
```

Таблицы и расчёты

3.11 Этап 11: Построение графика дискретной SIR-модели в веб-версии

Следующим шагом стало построение графика временных рядов $S(t)$, $I(t)$, $R(t)$ непосредственно в Jupyter-ноутбуке.

С использованием библиотеки StatsPlots были визуализированы изменения численности восприимчивых, инфицированных и переболевших во времени.

Такой график позволяет наглядно увидеть развитие эпидемического процесса, момент пика инфицированных и постепенный рост числа переболевших.

В веб-версии график отображается прямо под соответствующей ячейкой кода, что делает анализ результатов особенно удобным ([рис. fig-011?]).

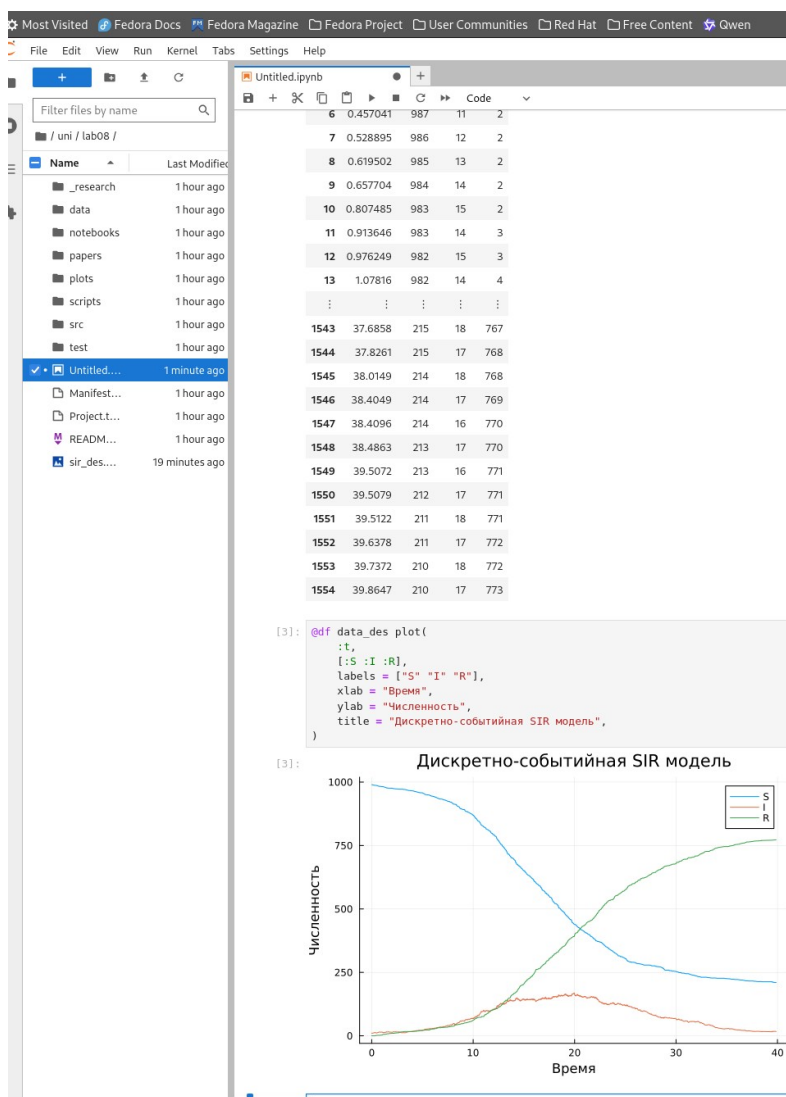


График дискретной SIR в веб-версии

3.12 Этап 12: Построение графика числа инфицированных при разных параметрах β

Для анализа чувствительности модели к параметру β был реализован цикл, который выполнял несколько прогонов симуляции при разных значениях вероятности заражения.

Результаты были объединены в единую таблицу и визуализированы в виде графика зависимости числа инфицированных $I(t)$ для различных β . Такой график показывает, как увеличение или уменьшение β влияет на высоту и положение пика эпидемии, а также на общую длительность вспышки.

Этот этап демонстрирует важность параметров модели и позволяет сделать выводы о влиянии контактной интенсивности и вероятности заражения на динамику процесса ([рис. fig-012?]).

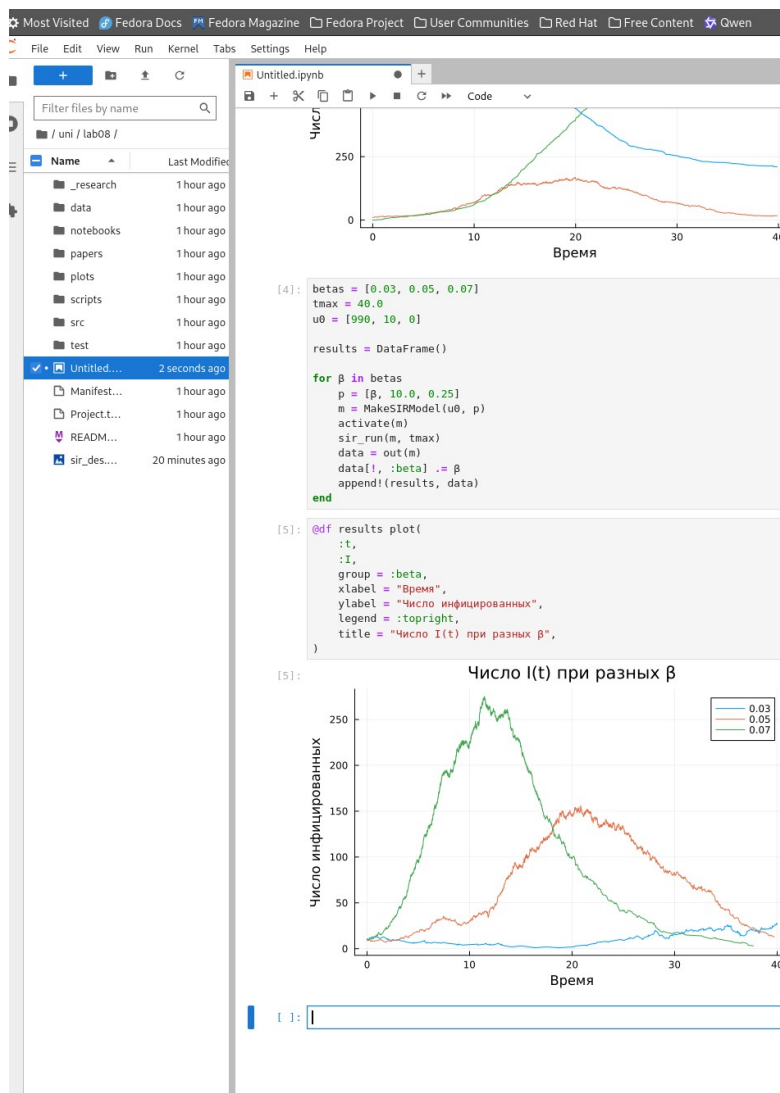


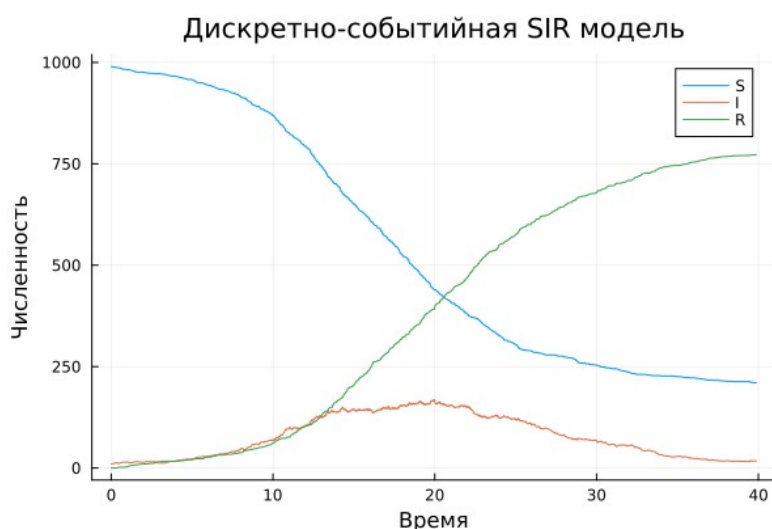
График $I(t)$ при разных β

3.13 Этап 13: Итоговая дискретно-событийная модель

Завершающим этапом стала фиксация итогового варианта дискретно-событийной SIR-модели и её интеграция в отчёт.

В результате была получена реализация, включающая описание модели, код ядра, скрипт запуска, Jupyter-ноутбук в литературном стиле и набор графиков.

Такая модель позволяет исследовать стохастическую динамику распространения инфекции в конечной популяции, учитывать случайные флуктуации и анализировать влияние параметров на поведение системы. Все полученные материалы были включены в отчёт и могут быть использованы как основа для дальнейших модификаций модели, например добавления демографических процессов или вакцинации ([рис. fig-013?]).



Итоговая дискретно-событийная модель

3.14 Вывод

В ходе лабораторной работы было освоено развертывание программного окружения для дискретно-событийного моделирования на языке Julia, включая установку самого интерпретатора, необходимых библиотек и интеграцию с Jupyter Notebook в литературном стиле.

Это позволило не только автоматизировать запуск модели, но и оформить вычислительный эксперимент в виде единого воспроизводимого документа.

Реализованная SIR-модель продемонстрировала типичное поведение эпидемического процесса в стохастической постановке: рост числа инфицированных до пика, последующее снижение и накопление переболевших.

Благодаря дискретно-событийному подходу удалось отразить влияние случайных контактов и флуктуаций, что отличает модель от

детерминированной системы ОДУ и делает результаты ближе к реальным сценариям.

Дополнительно был проведён анализ чувствительности к параметру β , показавший, что увеличение вероятности заражения приводит к более раннему и высокому пику числа инфицированных, а также к росту общей доли переболевших.

Это подтвердило важность корректного выбора параметров модели и продемонстрировало, как имитационное моделирование можно использовать для изучения влияния управляемых параметров на динамику эпидемии.

В целом, поставленные цель и задачи работы достигнуты: окружение настроено, модель реализована, результаты визуализированы и проанализированы, а полученный ноутбук может служить основой для дальнейших модификаций и расширений модели.